

Работа с базами данных. DataSet

Доцент каф. Евдошенко О.И.

Операция Cast и ofType

Операция Cast используется для приведения каждого элемента входной последовательности в выходную последовательность указанного типа.

Операция OfType используется для построения выходной последовательности, содержащей только те элементы, которые могут быть успешно преобразованы к указанному типу.

```
ArrayList al = new ArrayList();  
al.Add(new Employee { id = 1, firstName = "Joe", lastName = "Rattz" });  
al.Add(new Employee { id = 2, firstName = "William", lastName = "Gates" });  
al.Add(new Customer { id = 1, countbuy = 0 });  
al.Add(new Customer { id = 2, countbuy = 22 });  
al.Add(new Employee { id = 3, firstName = "Anders", lastName = "Hejlsberg" });  
al.Add(new Customer { id = 3, countbuy = 12 });
```

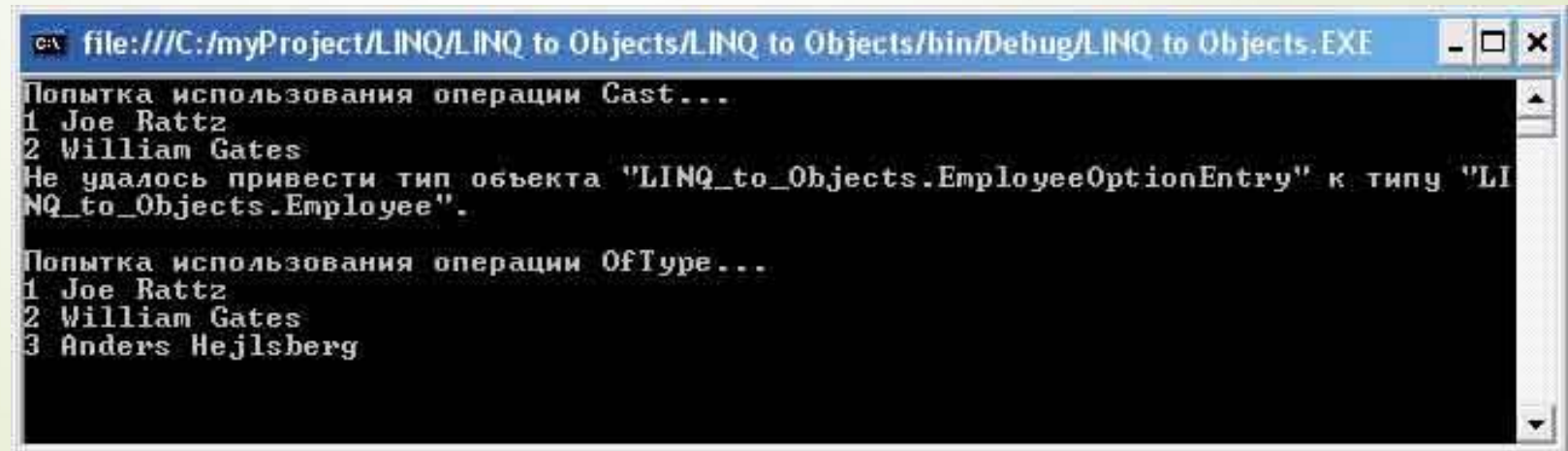
Операция Cast и ofType

```
var items = al.Cast<Employee>();
```

```
Console.WriteLine("Попытка использования операции Cast...");  
try  
{  
    foreach (Employee item in items)  
        Console.WriteLine("{0} {1} {2}", item.id, item.firstName, item.lastName);  
}
```

```
Console.WriteLine("Попытка использования операции OfType...");  
var items2 = al.Of<Employee>();
```

```
foreach (Employee item in items2)  
    Console.WriteLine("{0} {1} {2}", item.id, item.firstName, item.lastName);
```



```
file:///C:/myProject/LINQ/LINQ to Objects/LINQ to Objects/bin/Debug/LINQ to Objects.EXE  
Попытка использования операции Cast...  
1 Joe Rattz  
2 William Gates  
Не удалось привести тип объекта "LINQ_to_Objects.EmployeeOptionEntry" к типу "LINQ_to_Objects.Employee".  
Попытка использования операции OfType...  
1 Joe Rattz  
2 William Gates  
3 Anders Hejlsberg
```

DataSet и DataTable

```
DataSet bookStore = new DataSet("BookStore");
    DataTable booksTable = new DataTable("Books");
    // добавляем таблицу в dataset
    bookStore.Tables.Add(booksTable);

// создаем столбцы для таблицы Books
    DataColumn idColumn = new DataColumn("Id", Type.GetType("System.Int32"));
    idColumn.Unique = true; // столбец будет иметь уникальное значение
    idColumn.AllowDBNull = false; // не может принимать null
    idColumn.AutoIncrement = true; // будет автоинкрементироваться
    idColumn.AutoIncrementSeed = 1; // начальное значение
    idColumn.AutoIncrementStep = 1; // приращении при добавлении новой строки

    DataColumn nameColumn = new DataColumn("Name", Type.GetType("System.String"));
    DataColumn priceColumn = new DataColumn("Price", Type.GetType("System.Decimal"));
    priceColumn.DefaultValue = 100; // значение по умолчанию
    DataColumn discountColumn = new DataColumn("Discount", Type.GetType("System.Decimal"));
    discountColumn.Expression = "Price * 0.2";
```

DataSet и DataTable

```
booksTable.Columns.Add(idColumn);
    booksTable.Columns.Add(nameColumn);
    booksTable.Columns.Add(priceColumn);
    booksTable.Columns.Add(discountColumn);
    // определяем первичный ключ таблицы books
    booksTable.PrimaryKey = new DataColumn[] { booksTable.Columns["Id"] };

    DataRow row = booksTable.NewRow();
    row.ItemArray = new object[] { null, "Война и мир", 800 };
    booksTable.Rows.Add(row); // добавляем первую строку
    booksTable.Rows.Add(new object[] { null, "Отцы и дети", 370 }); // добавляем вторую строку

    dataGridView1.DataSource = bookStore.Tables[0];
```

```
1 | DataRow row = booksTable.Rows[0]; // первая строка
```

```
1 | booksTable.Rows[0][2] = 300; //третьей ячейке первой строки присваивается значение 300
```

```
1 | booksTable.Rows.RemoveAt(1); // удаление второй строки по индексу
2 | // другой способ удаления
3 | DataRow row = booksTable.Rows[0];
4 | booksTable.Rows.Remove(row);
```

```
1 | var selectedBooks = booksTable.Select("Price > 120");
2 | foreach (var b in selectedBooks)
3 |     Console.WriteLine("{0} - {1}", b["Name"], b["Price"]);
```

Отношения между таблицами в DataSet

```
DataTable companiesTable = new DataTable("Companies");
DataColumn compIdColumn = new DataColumn("Id", Type.GetType("System.Int32"));
    compIdColumn.Unique = true;
    compIdColumn.AllowDBNull = false;
    compIdColumn.AutoIncrement = true;
    compIdColumn.AutoIncrementSeed = 1;
    compIdColumn.AutoIncrementStep = 1;
DataColumn compNameColumn = new DataColumn("Name", Type.GetType("System.String"));
companiesTable.Columns.Add(compIdColumn);
companiesTable.Columns.Add(compNameColumn);
ds.Tables.Add(companiesTable);
```

```
        DataTable phonesTable = new DataTable("Phones");
            DataColumn phoneIdColumn = new DataColumn("Id", Type.GetType("System.Int32"));
            phoneIdColumn.Unique = true;
            phoneIdColumn.AllowDBNull = false;
            phoneIdColumn.AutoIncrement = true;
            phoneIdColumn.AutoIncrementSeed = 1;
            phoneIdColumn.AutoIncrementStep = 1;
            DataColumn phoneNameColumn = new DataColumn("Name", Type.GetType("System.String"));
            DataColumn phonePriceColumn = new DataColumn("Price", Type.GetType("System.Decimal"));
            // столбец-внешний ключ
            DataColumn idCompanyColumn = new DataColumn("CompanyId", Type.GetType("System.Int32"));

            phonesTable.Columns.Add(phoneIdColumn);
            phonesTable.Columns.Add(phoneNameColumn);
            phonesTable.Columns.Add(phonePriceColumn);
            phonesTable.Columns.Add(idCompanyColumn);
            ds.Tables.Add(phonesTable);
```

Отношения между таблицами в DataSet

```
// установка отношений между таблицами
ds.Relations.Add("PhonesCompanies",
companiesTable.Columns["Id"],
phonesTable.Columns["CompanyId"]);
```

Первым параметром здесь идет название отношения, по которому потом на него можно будет ссылаться. Вторым параметром идет главный столбец - id компании в таблице компаний. Третий параметр - подчиненный столбец - id компании в таблице смартфонов.

```
DataRow apple = companiesTable.NewRow();
apple.ItemArray = new object[] { null, "Apple" };
companiesTable.Rows.Add(apple);
```

```
DataRow iphone7 = phonesTable.NewRow();
iphone7.ItemArray = new object[] { null, "iPhone 7", 40000, apple["Id"] };
phonesTable.Rows.Add(iphone7);
```

```
DataRow samsung = companiesTable.NewRow();
samsung.ItemArray = new object[] { null, "Samsung" };
companiesTable.Rows.Add(samsung);
```

```
DataRow galaxy6 = phonesTable.NewRow();
galaxy6.ItemArray = new object[] { null, "Samsung Galaxy S6", 50000, samsung["Id"] };
phonesTable.Rows.Add(galaxy6);
```

```
// выведем все смартфоны компании Apple
```

```
DataRow[] rows = apple.GetChildRows(ds.Relations["PhonesCompanies"]);
```

GetChildRows – возвращает дочерние строки на основании указанного отношения.

```
foreach (DataRow r in rows)
{
    dataGridView1.Rows.Add(new object[] { r["Id"], r["Name"], r["Price"] });
}
```


Отношения между таблицами в DataSet

Кроме простого отношения между таблицами мы также можем задать внешние ключи

```
ForeignKeyConstraint foreignKey = new ForeignKeyConstraint(companiesTable.Columns["Id"], phonesTable.Columns["CompanyId"])
{
    ConstraintName = "PhonesCompaniesForeignKey",
    DeleteRule = Rule.SetNull,
    UpdateRule = Rule.Cascade
};
// добавляем внешний ключ в dataset
ds.Tables["Phones"].Constraints.Add(foreignKey);
// применяем внешний ключ
ds.EnforceConstraints = true;

ds.Relations.Add("PhonesCompanies", companiesTable.Columns["Id"], phonesTable.Columns["CompanyId"]);
```

Через свойство **ConstraintName** устанавливается название внешнего ключа. Свойства **DeleteRule** и **UpdateRule** определяет поведение при удалении или обновлении данных соответственно в главной таблице (то есть в данном случае в таблице компаний). Эти свойства принимают одно из значений перечисления **Rule**:

Cascade: происходит автоматическое изменение или удаление строк в подчиненной таблице

None: строки в подчиненной таблице не изменяются

SetDefault: происходит установка значений по умолчанию для связанных строк (которое определяется через DataColumn.DefaultValue)

SetNull: для связанных строк устанавливается значение DBNull

LINQ to DataSet

```
var query = from phone in ds.Tables["Phones"].AsEnumerable()  
            from company in ds.Tables["Companies"].AsEnumerable()  
            where (int)phone["CompanyId"] == (int)company["Id"]  
            where (decimal)phone["Price"] > 200  
            select new { Model = phone["Name"], Price = phone["Price"], Company = company["Name"] };
```

Чтобы избежать необходимости преобразований в запросах, можно использовать типизированный метод `Field<T>()`, в который передается название столбца, для которого надо получить значение:

```
var query = from phone in ds.Tables["Phones"].AsEnumerable()  
            from company in ds.Tables["Companies"].AsEnumerable()  
            where phone.Field<Int32>("CompanyId") == company.Field<Int32>("Id")  
            where phone.Field<Decimal>("Price") > 200  
            select new { Model = phone.Field<string>("Name"), Price = phone["Price"], Company = company["Name"] };
```

DataSet и XML

Встроенные возможности DataSet позволяют производить некоторые операции с xml-документами, в частности, сохранять данные из dataset в xml и, наоборот, загружать из xml в DataSet. Подобные возможности позволяют при необходимости использовать dataset в качестве посредника между базой данных и xml-файлами.

```
ds.WriteXml("d:\\1.xml");  
ds.Tables[0].WriteXml("d:\\1.xml");
```

```
<?xml version="1.0" standalone="yes"?>  
<NewDataSet>  
  <Companies>  
    <Id>1</Id>  
    <Name>Apple</Name>  
  </Companies>  
</NewDataSet>
```

```
DataSet ds_user = new DataSet();  
ds_user.ReadXml("d:\\1.xml");  
// выбираем первую таблицу  
DataTable dt_user = ds_user.Tables[0];  
foreach (DataColumn column in dt_user.Columns)  
  Console.WriteLine(column.ColumnName);  
Console.WriteLine();  
// перебор всех строк таблицы  
foreach (DataRow row in dt_user.Rows)  
{  
  var cells = row.ItemArray;  
  foreach (object cell in row.ItemArray)  
    Console.WriteLine(cell);  
  Console.WriteLine();  
}
```

```
<Users>  
  <User>  
    <Id>2</Id>  
    <Name>Петр</Name>  
    <Age>31</Age>  
  </User>  
  <User>  
    <Id>3</Id>  
    <Name>Ангелина</Name>  
    <Age>24</Age>  
  </User>  
</Users>
```

LINQ to SQL

Для взаимодействия с базой данных в LINQ to SQL используются модели и контекст данных. Модели представляют собой обычные классы, которые сопоставляются с одной из таблиц в базе данных.

```
using System.Data.Linq;  
using System.Data.Linq.Mapping;
```

Сущность для работы с данными из таблицы

```
[Table(Name = "Users")]  
public class User  
{  
    [Column(IsPrimaryKey = true)]  
    public int Id { get; set; }  
    [Column(Name = "Name")]  
    public string FirstName { get; set; }  
    [Column]  
    public int Age { get; set; }  
}
```

Атрибут **[Table]** позволяет выполнить сопоставление таблицы из БД с моделью. Однако по умолчанию модель должна называться также, как и таблица, то есть Users. Поэтому, чтобы сопоставление прошло успешно, в атрибуте Table указывается свойство Name с именем таблицы.

Каждое свойство модели соответствует столбцу по типу данных. Чтобы происходило автоматическое сопоставление, над свойствами применяется атрибут **[Column]**. Без данного атрибута система не сможет связать столбец из таблицы со свойством.

LINQ to SQL

Атрибут **[Column]** имеет ряд свойств, с помощью которого можно настроить сопоставление столбца:

AutoSync: указывает, как надо извлекать значение столбца после вставки или обновления

CanBeNull: указывает, может ли столбец принимать значение null

DbType: определяет тип столбца. Указывается, если надо создать новую базу данных

Expression: хранит выражение, которое будет использоваться для вычисления значения свойства

IsPrimaryKey: хранит логическое значение и указывает, выполняет ли столбец роль первичного ключа (как в данном случае Id)

IsDbGenerated: хранит логическое значение, которое указывает, будет ли значение столбца генерироваться самой БД

IsDiscriminator: указывает, будет ли столбец разграничителем в системе наследования классов

IsVersion: указывает, будет ли столбец хранить номер версии строки или значение timestamp, которое указывает на время последнего изменения строки

Name: задает имя столбца, с которым будет сопоставляться данное свойство

Storage: указывает на имя приватной переменной, которая будет хранить значение данного столбца

UpdateCheck: определяет, как LINQ to SQL будет решать проблему параллелизма. Если в модели нет свойств со значение IsVersion=true, то операциях с данными БД будет сравнивать значения строк из таблицы со новыми значениями.

LINQ to SQL

```
static string connectionString = @"Data Source=.\SQLEXPRESS;Initial Catalog=usersdb;Integrated Security=True";
static void Main(string[] args)
{
    DataContext db = new DataContext(connectionString);

    // Получаем таблицу пользователей
    Table<User> users = db.GetTable<User>();

    foreach (var user in users)
    {
        Console.WriteLine("{0} \t{1} \t{2}", user.Id, user.FirstName, user.Age);
    }

    Console.Read();
}
```

Операции с набором в LINQ to SQL

Фильтрация и сортировка

```
DataContext db = new DataContext(connectionString);
```

```
var query = from u in db.GetTable<User>()  
            where u.Age > 25  
            orderby u.FirstName  
            select u;
```

```
// или так
```

```
// var query = db.GetTable<User>().Where(u => u.Age > 25).OrderBy(u => u.FirstName);
```

```
foreach (var user in query)  
{  
    Console.WriteLine("{0} \t{1} \t{2}", user.Id, user.FirstName, user.Age);  
}
```

```
SELECT [t0].[Id], [t0].[Name] AS [FirstName], [t0].[Age]  
FROM [Users] AS [t0]  
WHERE [t0].[Age] > 25  
ORDER BY [t0].[Name]
```

Операции с набором в LINQ to SQL

Группировка

```
DataContext db = new DataContext(connectionString);
```

```
var query = from u in db.GetTable<User>()
             group u by u.Age // группировка по возрасту
             into grouped
             select grouped;
// или так
// var query = db.GetTable<User>().GroupBy(u => u.Age);
foreach (var group in query)
{
    Console.WriteLine("Возраст: {0}", group.Key);
    foreach (var user in group)
        Console.WriteLine(user.FirstName);
    Console.WriteLine();
}
```

```
SELECT [t0].[Age] AS [Key]
FROM [Users] AS [t0]
GROUP BY [t0].[Age]
```


Изменение объектов в LINQ to SQL

При получении объектов из базы данных в контекст **DataContext** эти объекты кэшируются, и у них устанавливается состояние **Unchanged**. Если мы изменим значения свойств какого-либо объекта из полученного набора, то DataContext для этого объекта создает копию с измененными значениями и устанавливает у нее статус ToBeUpdated.

При вызове метода **SubmitChanges()** контекст данных сверяет значения оригинального объекта и его измененной копии. И если два объекта отличаются, то создается sql-выражение UPDATE, с помощью которого происходит обновление объекта в базе данных.

```
// возьмем первого пользователя
User user1 = db.GetTable<User>().FirstOrDefault();
// и изменим у него возраст
user1.Age = 28;
// сохраним изменения
db.SubmitChanges();
```

Добавление в LINQ to SQL

Чтобы добавить новый объект в базу данных, необходимо вызвать у таблицы в контексте данных метод **InsertOnSubmit()** или **InsertAllOnSubmit()**(если надо добавить список объектов).

```
// создаем нового пользователя
User user1 = new User { FirstName = "Зиночка", Age = 34 };
// добавляем его в таблицу Users
db.GetTable<User>().InsertOnSubmit(user1);
db.SubmitChanges();
```

Данное выражение возвращает id добавленной записи, поэтому после добавления у объекта User можно получить Id:

```
Console.WriteLine(user1.Id);
```

Если в базе данных значение какого-либо столбца, например, столбца для Id, должно генерироваться автоматически, то в классе модели атрибут [**Column**] над соответствующим свойством должен иметь значение `IsDbGenerated = true`:

```
[Column(IsPrimaryKey = true, IsDbGenerated = true)]
public int Id { get; set; }
```

Основы валидации модели

Большую роль в приложении играет валидация модели или проверка вводимых данных на корректность. Например, у нас есть класс пользователя, в котором определено свойство для хранения возраста. И нам было бы нежелательно, чтобы пользователь вводил какое-либо отрицательное число или заведомо невозможный возраст

```
[Range(1, 100)]  
public int Age { get; set; }
```

Вначале мы создаем контекст валидации - объект **ValidationContext**. В качестве первого параметра в конструктор этого класса передается валидируемый объект, то есть в данном случае объект **User**.

Собственно валидацию производит класс **Validator** и его метод **TryValidateObject()**. В этот метод передается валидируемый объект (в данном случае объект **user**), контекст валидации, список объектов **ValidationResult** и булевый параметр, указывающий, надо ли валидировать все свойства.

Если метод **Validator.TryValidateObject()** возвращает **false**, значит объект не проходит валидацию. Если модель не проходит валидацию, то список объектов **ValidationResult** оказывается заполненным. А каждый объект **ValidationResult** содержит информацию о возникшей ошибке. Класс **ValidationResult** имеет два ключевых свойства:

MemberNames - список свойств, для которых возникла ошибка

ErrorMessage - собственно сообщение об ошибке.

```
// создаем нового пользователя  
User user1 = new User {Id=3, FirstName = "Зиночка", Age = -34 };  
// добавляем его в таблицу Users  
var results = new List<ValidationResult>();  
var context = new ValidationContext(user1);  
if (!Validator.TryValidateObject(user1, context, results, true))  
{  
    foreach (var error in results)  
    {  
        Console.WriteLine(error.ErrorMessage);  
    }  
}  
  
else  
{  
    db.GetTable<User>().InsertOnSubmit(user1);  
    db.SubmitChanges();  
}
```

Основы валидации модели. Атрибуты

Range: задает диапазон допустимых числовых значений. В качестве первых двух параметров он принимает минимальное и максимальное значения.

```
[Range(1, 100, ErrorMessage = "Недопустимый возраст")]  
public int Age { get; set; }
```

Required: данный атрибут указывает, что свойство должно быть обязательно установлено, обязательно должно иметь какое-либо значение. Из его свойств следует отметить свойство `AllowEmptyStrings`. Если оно имеет значение `true`, то для строковых свойств разрешено использовать пустые строки.

RegularExpression: указывает на регулярное выражение, которому должно соответствовать значение свойства. Например, пусть у пользователя определено свойство `Phone`, хранящее номер телефона:

```
[RegularExpression(@"^\+[1-9]\d{3}-\d{3}-\d{4}$",  
ErrorMessage = "Номер телефона должен иметь формат +xxxx-xxx-xxxx")]
```

StringLength: определяет допустимую длину для строковых свойств. В качестве первого параметра он принимает максимально допустимую длину строки. С помощью дополнительного свойства `MinimumLength` можно установить минимально допустимую длину строки.

Основы валидации модели. Атрибуты

Compare: позволяет сравнить значение текущего свойства со значением другого свойства, которое передается в этот атрибут.

```
public string Password { get; set; }  
[Required]  
[Compare("Password")]  
public string ConfirmPassword { get; set; }
```

Phone: данный атрибут автоматически валидирует значение свойства, является ли оно телефонным номером. Фактически это встроенная альтернатива использованию регулярного выражения, как было показано выше

EmailAddress: определяет, является ли значение свойства электронным адресом

CreditCard: определяет, является ли значение свойства номером кредитной карты

Url: определяет, является ли значение свойства гиперссылкой

Создание своих атрибутов валидации

```
[UserName]  
public string FirstName { get; set; }
```

```
public class UserNameAttribute : ValidationAttribute  
{  
    public override bool IsValid(object value)  
    {  
        if (value != null)  
        {  
            string userName = value.ToString();  
            if (!userName.StartsWith("T"))  
                return true;  
            else  
                this.ErrorMessage = "Имя не должно начинаться с буквы Т";  
        }  
        return false;  
    }  
}
```

```
[Table(Name = "User")]  
[UserValidation]  
public class User  
{  
    .....  
}
```

```
User user = value as User;  
if (user.Age-user.Staj <= 18)  
{  
    this.ErrorMessage = "Разница между возрастом и стажем должна быть неменьше 18 лет";  
    return false;  
}  
return true;
```

Удаление в LINQ to SQL

```
var user = db.GetTable<User>().OrderByDescending(u => u.Id).FirstOrDefault();
```

```
if (user != null)
{
    Console.WriteLine("Удаляемый объект:");
    Console.WriteLine("Id: {0}", user.Id);
    Console.WriteLine("Имя: {0}", user.FirstName);
    Console.WriteLine("Возраст: {0}", user.Age);

    db.GetTable<User>().DeleteOnSubmit(user);
    db.SubmitChanges();
    Console.WriteLine("Объект удален");
}
```

У данного способа есть большой недостаток: если необходимо удалить объект по какому-либо признаку, например, по ID, то вначале нам надо получить этот объект и потом уже передать его в метод **DeleteOnSubmit()**. При этом будут происходить фактически два обращения к БД: на получение объекта и на его удаление. Чтобы избежать ненужного запроса можно использовать хранимые процедуры или прямые запросы SQL.

Методы ExecuteCommand и ExecuteQuery

Для непосредственного выполнения кода SQL в классе DataContext определены методы **ExecuteCommand()** и **ExecuteQuery()**.

Метод **ExecuteCommand()** принимает выполняемое SQL-выражение и список параметров:

```
1 string connectionString = @"Data Source=.\SQLEXPRESS;Initial Catalog=usersdb;Integrated Security=True";
2 DataContext db = new DataContext(connectionString);
3
4 int modifiedRowsNumber = db.ExecuteCommand("DELETE FROM Users WHERE Id={0}", 40);
```

Возвращаемое методом число указывает на количество затронутых командой строк.

Метод **ExecuteQuery()** возвращает результат выполнения запроса SELECT:

```
1 DataContext db = new DataContext(connectionString);
2 IEnumerable<User> users = db.ExecuteQuery<User>("SELECT * FROM Users WHERE Age>{0}", 23);
```

Что такое Entity Framework

Entity Framework представляет специальную объектно-ориентированную технологию на базе фреймворка .NET для работы с данными. Если традиционные средства ADO.NET позволяют создавать подключения, команды и прочие объекты для взаимодействия с базами данных, то Entity Framework представляет собой более высокий уровень абстракции, который позволяет абстрагироваться от самой базы данных и работать с данными независимо от типа хранилища.

Центральной концепцией Entity Framework является понятие **сущности** или entity. Сущность представляет набор данных, ассоциированных с определенным объектом. Поэтому данная технология предполагает работу не с таблицами, а с объектами и их наборами.

Другим ключевым понятием является **Entity Data Model**. Эта модель сопоставляет классы сущностей с реальными таблицами в БД.

Entity Data Model состоит из трех уровней: концептуального, уровень хранилища и уровень сопоставления (маппинга).

На концептуальном уровне происходит определение классов сущностей, используемых в приложении.

Уровень хранилища определяет таблицы, столбцы, отношения между таблицами и типы данных, с которыми сопоставляется используемая база данных.

Уровень сопоставления (маппинга) служит посредником между предыдущими двумя, определяя сопоставление между свойствами класса сущности и столбцами таблиц.

Что такое Entity Framework

Способы взаимодействия с БД

Entity Framework предполагает три возможных способа взаимодействия с базой данных:

- **Database first:** Entity Framework создает набор классов, которые отражают модель конкретной базы данных
- **Model first:** сначала разработчик создает модель базы данных, по которой затем Entity Framework создает реальную базу данных на сервере.
- **Code first:** разработчик создает класс модели данных, которые будут храниться в БД, а затем Entity Framework по этой модели генерирует базу данных и ее таблицы

Отличительной чертой Entity Framework является использование запросов LINQ для выборки данных из БД. С помощью LINQ мы можем не только извлекать определенные строки, хранящие объекты, из БД, но и получать объекты, связанные различными ассоциативными связями.

Автоматизация Code First

Добавление нового элемента - WindowsFormsApp1

Установленные

Сортировка: По умолчанию

Поиск (Ctrl+E)

Элементы Visual C#

WPF

Code

Data

General

SQL Server

Web

Windows Forms

В сети

XML-файл

Элементы Visual C#

XSLT-файл

Элементы Visual C#

База данных, основанная на службах

Элементы Visual C#

Генератор EF 5.x DbContext

Элементы Visual C#

Генератор EF 6.x DbContext

Элементы Visual C#

Классы LINQ to SQL

Элементы Visual C#

Модель ADO.NET EDM

Элементы Visual C#

Набор данных

Элементы Visual C#

Схема XML

Элементы Visual C#

Тип: Элементы Visual C#

Элемент проекта для создания модели EDM ADO.NET.

Имя:

Sotrudniki

Добавить

Отмена

Мастер моделей EDM

Выбор содержимого модели

Что должна содержать модель?

Конструктор EF из базы данных

Пустая модель конструктора EF

Пустая модель Code First

Code First из базы данных

Создает модель Code First на основе существующей базы данных. Можно выбрать подключение к базе данных, параметры для модели и объекты базы данных для включения в модель.

Мастер моделей EDM

Выбор подключения к данным

Какое подключение к данным будет использоваться приложением для подключения к базе данных?

ГородаConnectionString (Settings)

Создать соединение...

Возможно, эта строка подключения содержит конфиденциальные данные (например, пароль), которые требуются для подключения к базе данных. Хранение конфиденциальных данных в строке подключения может представлять угрозу безопасности. Включить конфиденциальные данные в строку подключения?

Нет, исключить конфиденциальные данные из строки подключения. Они будут заданы в коде приложения.

Да, включить конфиденциальные данные в строку подключения.

Строка подключения:
data source=(LocalDB)\MSSQLLocalDB;attachdbfilename=|DataDirectory|\Города.mdf;integrated security=True;MultipleActiveResultSets=True;App=EntityFramework

Сохранить параметры соединения в App.Config как:

Sotrudniki

< Назад

Далее >

Готово

Отмена

Автоматизация Code First

Мастер моделей EDM

Выберите параметры и объекты базы данных

Какие объекты базы данных нужно включить в модель?

Таблицы

dbo

Dol

Sot

Users

Представления

☐ Формировать имена объектов во множественном или единственном числе

☒ Включить столбцы внешних ключей в модель

☐ Импортировать выбранные хранимые процедуры и функции в модель сущностей

< Назад

Далее >

Готово

Отмена

Результаты				
	ID	FIO	Staj	Dol
1	1	Иванов	300	1
2	2	Петров	10	1
3	3	Егоров	5	2
4	4	Том	33	2
5	5	Коробочкин	2	1

Результаты		
	ID	Nazvanie
1	1	директор
2	2	инженер

Соглашения по наименованию в Code First

NULL и NOT NULL

Все первичные ключи по умолчанию имеют определение NOT NULL.

Столбцы, сопоставляемые со свойствами ссылочных типов (string, array), в базе данных имеют определение NULL, а все значимые типы (DateTime, bool, char, decimal, int, double, float) - NOT NULL.

Если свойство имеет тип Nullable<T>, то оно сопоставляется со столбцом с определением NULL.

Ключи

Entity Framework требует наличия первичного ключа, так как это позволяет ему отслеживать объекты. По умолчанию в качестве ключей EF рассматривает свойства с именем Id или [Название_типа]Id (например, PostId в классе Post).

Как правило, ключи имеют тип int или GUID, но также могут представлять и любой другой примитивный тип.

Автоматизация Code First

```
[Table("Sot")]
public partial class Sot
{
    [DatabaseGenerated(DatabaseGeneratedOption.None)]
    public int ID { get; set; }

    [StringLength(50)]
    public string FIO { get; set; }

    public int? Staj { get; set; }

    public int? Dol { get; set; }
}
```

Класс контекста данных:

```
public partial class Sotrudniki : DbContext
{
    public Sotrudniki()
        : base("name=Sotrudniki")
    {
    }

    public virtual DbSet<Dol> Dol { get; set; }
    public virtual DbSet<Sot> Sot { get; set; }
}
```

Взаимодействие с базой данных:

```
using (Sotrudniki db = new Sotrudniki())
{
    foreach (Sot u in db.Sot)
        Console.WriteLine("{0}.{1} - {2}", u.ID, u.FIO, u.Staj);
}
```

Сопоставление типов

Типы SQL Server и C# сопоставляются следующим образом:

- int : int
- bit : bool
- char : string
- date : DateTime
- datetime : DateTime
- datetime2 : DateTime
- decimal : decimal
- float : double
- money : decimal
- nchar : string
- ntext : string
- numeric : decimal
- nvarchar : string
- real : float
- smallint : short
- text : string
- tinyint : byte
- varchar : string